W3.io Protocol

Verifiable Workflow Execution
with On-Chain Settlement

Version 2.0

Audie Sheridan, Ben Murray, David Post

w3.io

2026

Contents

1 Abstract

W3.io is a workflow execution protocol that connects fragmented Web3 infrastructure into verifiable, multi-step workflows that power real-world applications. Developers compose workflows from modular actions — blockchain operations across Ethereum, Solana, and Bitcoin, data queries, AI inference, payments, compliance checks, and dozens of other ecosystem partner capabilities — using a declarative YAML syntax compatible with GitHub Actions. The protocol transforms isolated infrastructure components into programmable, composable pipelines where each step's execution is attested by BFT consensus and provable against an on-chain commitment.

A distributed network of validators executes each workflow step, reaches Byzantine fault-tolerant consensus on the result, and produces cryptographic settlement receipts. These receipts are accumulated into epoch commitments anchored to an EVM L1, enabling any party to confirm that a

specific workflow execution was attested by a BFT quorum without re-executing the workflow or trusting any individual validator.

The W3 token gates participation in the network. Ecosystem partners post collateral to operate subnets, solution builders stake to deploy applications, sales teams stake to participate in adoption programs, and validators bond tokens to secure consensus. Workflow execution fees are denominated in stablecoins, separating the utility token from the payment medium.

This paper describes the architecture, workflow execution model, consensus mechanism, settlement layer, namespace model, validator infrastructure, token utility, and security properties of the W3.io protocol. The protocol is implemented in Rust, open-source, and deployed on testnet with over 200,000 workflow executions per day.

2 Key Contributions

W3.io makes five specific contributions to the state of decentralized infrastructure.

1. GitHub Actions as the workflow language. W3.io's workflow syntax is not a custom DSL. It is the same YAML grammar used by GitHub Actions, parsed by the same rules, supporting the same expression language. The developer population that already writes CI/CD workflows can write W3.io workflows with zero additional learning. No other decentralized execution protocol offers this on-ramp.

2. One consensus algorithm, four contexts. Rather than bolting different consensus mechanisms onto different protocol functions, W3.io uses a single BFT protocol for step runner selection, step attestation, trigger confirmation, and epoch manifest agreement. The algorithm is identical in each context. The proposal values and committee compositions differ. This uniformity simplifies security analysis: one protocol to audit, one set of properties to verify.

3. Per-execution partner attribution. Every workflow receipt includes an `actions_used` field: a sorted list of action IDs for every partner capability invoked during the run. This makes fee attribution, billing, and revenue allocation automatic by construction. There is no separate analytics pipeline or manual accounting. The settlement layer knows which partners contributed to every workflow execution.

4. Non-inclusion proofs. W3.io's sparse merkle tree supports proving that a workflow did NOT run. Most settlement layers can only prove what happened. W3.io can also prove what didn't. This is critical for dispute resolution ("you claim you ran my workflow; prove it against the on-chain root") and compliance ("prove that no unauthorized workflow executed in this namespace during this period").

5. Stablecoin fees with token collateral. Workflow execution fees are denominated in USDC.

The W3 token is used for staking and access control, not as a payment medium. This avoids the circular dependency where using the token for fees creates artificial demand that inflates costs for users. Businesses get predictable pricing. Token utility comes from participation requirements, not forced usage.

3 Introduction

3.1 The Integration Gap

Decentralized infrastructure has matured rapidly. Individual services now exist for payments, data, compute, storage, identity, oracles, and security. Multiple blockchains provide settlement for different use cases (Nakamoto 2008; Buterin 2014; Yakovenko 2018). The building blocks are there.

What doesn't exist is a way to connect these services into verifiable end-to-end workflows. A business process that requires a payment, a compliance check, a data query, and an on-chain settlement today requires custom integration code for each service, separate authentication and billing relationships with each provider, and no unified way to verify that the entire process executed correctly.

This gap has three dimensions.

Centralization by default. When developers need to orchestrate multiple services, the path of least resistance is a centralized server calling each API sequentially. This contradicts the decentralized properties of the underlying services, introduces single points of failure, and creates vendor lock-in with cloud providers. The result is decentralized infrastructure coordinated by centralized glue code.

Integration complexity. Each decentralized service has its own API patterns, authentication mechanisms, data formats, and error handling conventions. Integrating even two services requires specialized knowledge of both. Integrating five or ten, as real business processes demand, multiplies the development cost, the testing surface, and the ongoing maintenance burden. Most Web3 development is bespoke, built from scratch against the same patterns that every other team is also building from scratch.

Business friction. Beyond the technical challenges, using multiple decentralized providers means separate legal agreements, separate payment arrangements, and separate vendor relationships. Discovery is also a problem. Hundreds of decentralized projects exist, but their capabilities and integration surfaces are poorly documented and difficult to evaluate without deep technical engagement.

These challenges are not hypothetical. They are the reason most production Web3 applications still run their off-chain logic on AWS, even when the on-chain components are fully decentralized.

A DeFi protocol settles on Ethereum but runs its liquidation bots on EC2. A payments company uses Circle's on-chain USDC but orchestrates compliance checks through a Lambda function. The blockchain provides the settlement guarantee. A centralized server provides everything in between. W3.io replaces that centralized server with a protocol.

3.2 W3.io's Approach

W3.io addresses the integration gap with a protocol-level execution layer that connects decentralized services into verifiable workflows. Rather than requiring developers to write custom integration code, W3.io provides a declarative workflow language where each step invokes a pre-integrated action from the W3.io ecosystem.

W3.io's model has three layers.

Actions are modular capabilities provided by ecosystem partners. An action encapsulates a specific operation behind a standard interface: querying a database, executing a blockchain transaction, running an AI model, sending a payment. Actions are versioned, namespace-scoped, and independently deployable. W3.io ships with native actions for Ethereum, Solana, and Bitcoin blockchain operations. Partners contribute actions for their specific capabilities.

Workflows are sequences of actions triggered by events. A workflow definition specifies a trigger (a cron schedule, an RPC call, a blockchain event, or an oracle feed), a series of steps that each invoke an action, and the data flow between steps. Workflows are compiled from YAML into a canonical form, deployed to the validator network, and executed with Byzantine fault-tolerant consensus on each step's result.

Solutions are workflows integrated into end-user applications. A treasury management application, a compliance pipeline, a cross-chain bridge: each is a solution built from workflows that compose ecosystem actions. Solutions inherit the verifiability of the underlying protocol without requiring the end user to understand the execution mechanics.

This layered model means that ecosystem partners contribute capabilities once, developers compose them into workflows without custom integration code, and businesses deploy solutions that are verifiable by construction.

3.3 What This Paper Covers

The remainder of this paper describes the W3.io protocol in detail:

- **Architecture**: system overview, data flow, and the relationship between protocol components
- **Workflows**: the declarative syntax, trigger types, step kinds, and expression evaluation

- **Execution**: how steps are isolated and executed on validators, with timeout enforcement and signal escalation
- **Consensus**: the BFT protocol, committee selection, and the four contexts in which consensus is used
- **Settlement**: the cryptographic commitment structure that makes workflow attestations independently provable on-chain
- **Cryptography**: the specific algorithms used and their provenance in production-grade, audited implementations
- **Namespaces**: the hierarchical multi-tenancy model for organizing workflows, billing, and access control
- **Validators**: how operators join the network, declare capabilities, and maintain liveness
- **Ecosystem**: the B2B2C partner model and the action registry
- **Token**: the utility of the W3 token within the protocol and its economic structure
- **Security**: the threat model, BFT assumptions, and known limitations

W3.io's protocol is implemented in Rust (Matsakis and Klock 2014), open-source, and deployed on testnet. This paper describes the system as designed and implemented, not as aspirational. Where features are planned but not yet deployed, this is stated explicitly.

4 Architecture

4.1 Ingredients, Recipes, and Solutions

W3.io's ecosystem is organized around three tiers of composability. The terminology is borrowed from the kitchen, and the analogy is precise. A chef does not grow wheat or raise cattle. A chef combines high-quality ingredients into recipes that produce something none of the ingredients could deliver alone. The chef's skill is in the composition: knowing which ingredients work together, in what order, under what conditions. W3.io applies this model to decentralized infrastructure. Partners provide the ingredients. Developers write the recipes. Businesses serve the resulting solutions to their customers.

Ingredients are capabilities provided by individual ecosystem partners. Each partner exposes their capabilities as modular, reusable components within the W3.io ecosystem. Space and Time provides data queries. Circle and Stripe handle payments. Hyperbolic offers AI inference. Chainalysis handles compliance screening. Pyth delivers price feeds. An ingredient is a self-contained unit of functionality that can be invoked as part of a larger process.

Recipes combine two or more ingredients into adaptive logic that addresses a specific business need. A recipe defines the trigger conditions, the sequence of operations, the data flow between

steps, and the failure handling behavior. Recipes are W3.io's unit of deployment. A developer writes a recipe, deploys it to the validator network, and it executes automatically when triggered.

Solutions are recipes integrated into end-user applications. A payments product, a compliance pipeline, a yield vault: each is a solution that composes ingredients through recipes to deliver a complete user experience. Solutions inherit the verifiability of the underlying protocol. Every step of every recipe execution is attested by consensus and provable against an on-chain commitment.

This model enables a network effect. Each new ingredient makes every existing recipe potentially more capable, and each new recipe demonstrates a pattern that other builders can adapt.

4.2 Actions and Workflows

At the protocol level, ingredients become **actions** and recipes become **workflows**.

An action has a namespace-scoped identifier, a version, and defined inputs and outputs. W3.io ships with native actions for three blockchain families:

- **Ethereum**: balance queries, token transfers and approvals, contract deployment and calls, event monitoring, NFT operations, ENS name resolution
- **Solana**: balance queries, token accounts, program invocation, transfers, transaction monitoring
- **Bitcoin**: balance queries, UTXO management, fee estimation, transaction construction and broadcast

Ecosystem partners extend the action set by publishing their own actions. These are packaged as container images with a standard entry point, versioned independently, and invoked via the `uses:` step kind in workflow YAML. Current ecosystem actions include data queries (Space and Time), price feeds (Pyth), compliance screening (Chainalysis), payments (Circle, Stripe), AI inference (Hyperbolic), caching (Redis), email delivery, and token transfers.

Workflows are defined in YAML using a syntax compatible with GitHub Actions:

```yaml
name: Cross-chain settlement
on:
  schedule:
    - cron: '*/5 * * * *'
jobs:
  settle:
    steps:
      - name: Query pending transactions
        uses: w3-io/w3-sxt-action@v1
```

```
        with:
          command: query
          sql: >
            SELECT * FROM transactions
            WHERE status = 'pending'

      - name: Screen for compliance
        uses: w3-io/w3-chainalysis-action@v1
        with:
          address: ${{ steps.query.outputs.sender }}

      - name: Execute settlement
        ethereum:
          action: call-contract
          network: ethereum
          params:
            to: "0x..."
            function: "settle(address,uint256)"
            args:
              - ${{ steps.query.outputs.sender }}
              - ${{ steps.query.outputs.amount }}
```

The GitHub Actions compatibility is deliberate. Developers already familiar with CI/CD workflows can write W3.io workflows without learning a new language. The YAML is compiled into a canonical form, hashed for on-chain provenance, and deployed to the validator network.

4.3 The Three-Layer Data Architecture

W3.io separates data responsibilities across three layers, each optimized for its role.

Layer 1: L1 Settlement. The on-chain layer stores commitments, not data. The footprint is minimal: one cumulative root per epoch, plus contract state for the validator set and workflow registry. This keeps gas costs low while providing the anchor for cryptographic verification.

Layer 2: Protocol Nodes. Validators store everything. Full workflow execution traces, receipt storage, epoch accumulation, P2P consensus messages, and a write-ahead log for crash recovery. This layer is the evidence behind the Layer 1 commitments and serves as the data availability layer for proof generation.

Layer 3: Index and Query. An indexer watches Layer 1 events and reconstructs queryable

views: workflow runs by namespace, by epoch, by action type. The CLI provides developer access to settlement status and proof export. The API layer supports MCP (Model Context Protocol) for AI agent integration.

4.4 AI Agent Integration

W3.io is designed for a world where AI agents are first-class participants in financial workflows. The protocol exposes its capabilities through an MCP (Model Context Protocol) server, the emerging standard for connecting AI assistants to external tools and data.

The W3 MCP server gives any MCP-compatible AI agent the ability to compile and validate workflows, deploy workflows to the network, query workflow execution status, inspect settlement receipts and epoch data, and interact with the Solana DEX indexer and other data services. An AI agent using Claude, GPT, or any MCP-compatible model can discover W3.io's capabilities programmatically, invoke them through structured tool calls, and compose multi-step financial operations without custom integration code.

This is not a future feature. The W3 MCP server is published, operational, and registered in the MCP ecosystem. AI agents can deploy and trigger W3.io workflows today using the same natural language interface they use for any other MCP-connected tool.

The integration works in both directions. AI agents can orchestrate W3.io workflows (deploying recipes, triggering executions, reading results). And W3.io workflows can invoke AI capabilities as steps (via the Hyperbolic action for inference, or any AI provider that publishes an action). The protocol does not privilege human users over programmatic ones. A workflow triggered by an AI agent goes through the same consensus, attestation, and settlement pipeline as one triggered by a human.

4.5 The Spine and Ribs

W3.io's settlement architecture is best understood as a spine with ribs.

```
The Spine: L1 Epoch Chain

  Epoch 0        Epoch 1        Epoch 2
  root: 0xaaa    root: 0xbbb    root: 0xccc
  prev: 0x000    prev: 0xaaa    prev: 0xbbb

The Ribs: Per-Run Hashchains

  Workflow A -- block 1 -- block 2 -- receipt
```

```
Workflow B -- block 1 -- receipt
Workflow C -- block 1 -- block 2 -- block 3 -- receipt
```

The **spine** is the L1 epoch chain. A sequential, immutable record of cumulative sparse merkle tree roots (Gao et al. 2021) on-chain. Each epoch's root covers all workflow receipts across all namespaces ever settled, not just the current epoch. The chain of roots captures the full evolution of world state.

The **ribs** are individual workflow execution traces. Per-run hashchains of consensus blocks that branch off the spine. Each step in a workflow produces a block containing the step's inputs, outputs, executor identity, and the previous block's hash. When the workflow completes, the final block hash and metadata are compressed into a receipt. That receipt is inserted into the namespace's sparse merkle tree, which feeds into the global tree, which produces the epoch root committed to the spine.

Any workflow execution is provable against any epoch root that includes it. A verifier needs only the on-chain root, a receipt, and two merkle proofs (receipt-in-namespace, namespace-in-global) to confirm that a specific workflow executed with a specific result at a specific time. No re-execution required. No trust in the validator that ran it.

5 Workflows

5.1 Declarative Syntax

W3.io workflows are defined in YAML using the same syntax as GitHub Actions. This is not a loose compatibility. The W3.io compiler parses the same grammar, supports the same expression language, and handles the same structural elements: jobs, steps, conditionals, environment variables, secrets, outputs, and step references.

A developer who has written a GitHub Actions workflow can write a W3.io workflow without learning anything new. The difference is what happens after deployment. A GitHub Actions workflow runs on a centralized runner. A W3.io workflow runs on a distributed validator network with BFT consensus on every step.

The following example shows a non-trivial workflow that monitors an Ethereum wallet for large inbound transfers, screens the sender for sanctions compliance, logs the event to a database for audit, and sends a notification. It uses four ecosystem partners (Chainalysis for compliance, Space and Time for data, a Redis cache for deduplication, and an email action for alerting) alongside native Ethereum actions.

```yaml
name: Large transfer compliance monitor
on:
  schedule:
    - cron: '*/5 * * * *'
jobs:
  monitor:
    steps:
      - name: Check recent transfers
        id: transfers
        uses: w3-io/w3-sxt-action@v1
        with:
          command: query
          sql: >
            SELECT tx_hash, from_address, value
            FROM ethereum.erc20_transfers
            WHERE to_address = '${{ secrets.WATCHED_WALLET }}'
            AND value > 10000000000
            AND block_timestamp > NOW() - INTERVAL '5 MINUTES'

      - name: Deduplicate against cache
        id: dedup
        if: steps.transfers.outputs.row_count > 0
        uses: w3-io/w3-redis-action@v1
        with:
          command: sismember
          url: ${{ secrets.REDIS_URL }}
          key: processed-transfers
          value: ${{ steps.transfers.outputs.rows[0].tx_hash }}

      - name: Screen sender address
        id: compliance
        if: steps.dedup.outputs.result == '0'
        uses: w3-io/w3-chainalysis-action@v1
        with:
          address: ${{ steps.transfers.outputs.rows[0].from_address }}
```

```yaml
    - name: Log to audit trail
      if: steps.compliance.outputs.risk != 'sanctions'
      uses: w3-io/w3-sxt-action@v1
      with:
        command: execute
        sql: >
          INSERT INTO compliance_log
          (tx_hash, sender, amount, risk_level, checked_at)
          VALUES ('${{ steps.transfers.outputs.rows[0].tx_hash }}',
                  '${{ steps.transfers.outputs.rows[0].from_address }}',
                  ${{ steps.transfers.outputs.rows[0].value }},
                  '${{ steps.compliance.outputs.risk }}',
                  NOW())

    - name: Mark as processed
      uses: w3-io/w3-redis-action@v1
      with:
        command: sadd
        url: ${{ secrets.REDIS_URL }}
        key: processed-transfers
        value: ${{ steps.transfers.outputs.rows[0].tx_hash }}

    - name: Alert on high risk
      if: steps.compliance.outputs.risk == 'high'
      uses: w3-io/w3-email-action@v1
      with:
        to: compliance@example.com
        subject: >
          High risk transfer detected:
          ${{ steps.transfers.outputs.rows[0].tx_hash }}
```

Every step in this workflow executes on a different validator, selected by consensus. The compliance check result, the database write, and the alert are all attested by the committee and included in the settlement receipt. An auditor can later prove that the compliance check happened, what it returned, and when, by verifying the receipt against the on-chain epoch root.

A second example shows composability across four partners where money actually moves. A treasury application must disburse USDC to a recipient, but only after verifying the stablecoin peg is

stable, the recipient passes sanctions screening, and every decision is logged to an immutable audit trail.

```yaml
name: Compliant stablecoin disbursement
on:
  workflow_dispatch:
    inputs:
      recipient_address:
        description: 'Recipient wallet address'
        required: true
      amount_usd:
        description: 'Disbursement amount in USD'
        required: true
jobs:
  disburse:
    steps:
      - name: Check peg stability
        id: peg
        uses: w3-io/w3-pyth-action@v1
        with:
          feed: USDC/USD
          max_deviation_bps: 10

      - name: Screen recipient
        id: screening
        if: steps.peg.outputs.within_band == 'true'
        uses: w3-io/w3-chainalysis-action@v1
        with:
          address: ${{ inputs.recipient_address }}

      - name: Log decision to audit trail
        if: steps.screening.outputs.risk != 'sanctions'
        uses: w3-io/w3-sxt-action@v1
        with:
          command: execute
          sql: >
            INSERT INTO disbursement_log
```

```
            (recipient, amount_usd, risk_level, peg_rate)
            VALUES ('${{ inputs.recipient_address }}',
                    ${{ inputs.amount_usd }},
                    '${{ steps.screening.outputs.risk }}',
                    ${{ steps.peg.outputs.rate }})

      - name: Execute disbursement
        if: steps.screening.outputs.risk != 'sanctions'
        uses: w3-io/w3-circle-action@v1
        with:
          action: transfer
          destination: ${{ inputs.recipient_address }}
          amount: ${{ inputs.amount_usd }}
          currency: USDC
```

Before a single dollar moves, the workflow confirms the USDC peg is within 10 basis points of par (Pyth), verifies the recipient is not sanctions-listed (Chainalysis), writes the decision to a tamper-proof audit log (Space and Time), and only then executes the transfer (Circle). If any step fails, nothing moves. The settlement receipt proves not just that the payment happened, but that the peg check and sanctions screen happened first, in that order, with those results.

Each partner made this possible by contributing one ingredient. Space and Time did not need to know about Circle. Pyth did not need to know about Chainalysis. Each partner published their capability once. The recipe assembles them. When a fifth partner joins the ecosystem, every existing recipe can add a step without any of the other partners changing anything.

A workflow definition includes:

- **Name**: human-readable identifier for the workflow
- **Trigger** (`on:`): the event that initiates execution
- **Jobs**: one or more named job blocks, each containing a sequence of steps
- **Steps**: ordered operations within a job, each invoking an action or running a command
- **Environment** (`env:`): key-value pairs scoped to the workflow, job, or individual step
- **Secrets**: sensitive values fetched at runtime from a secrets provider, never stored in the workflow definition

5.2 Triggers

Every workflow begins with a trigger. W3.io supports four trigger types.

Cron schedule (`schedule`). Time-based triggers that fire on a cron expression. A workflow triggered every five minutes, every hour, or once a day. The cron expression follows standard POSIX syntax. Useful for batch processing, periodic data aggregation, and scheduled reporting.

```yaml
on:
  schedule:
    - cron: '*/5 * * * *'
```

RPC dispatch (`workflow_dispatch`). On-demand triggers fired by an API call. A user, an application, or an AI agent sends a request to the W3.io RPC endpoint with optional input parameters. The workflow runs once with the provided inputs. Useful for user-initiated actions like deploying a contract, processing a payment, or running an analysis.

```yaml
on:
  workflow_dispatch:
    inputs:
      token_address:
        description: 'Token to analyze'
        required: true
      chain:
        description: 'Blockchain network'
        default: 'ethereum'
```

Chain events. Blockchain event triggers that fire when a specific on-chain event occurs. An ERC-20 transfer, a contract deployment, a price threshold crossing. The validator network monitors the specified chain for matching events and initiates the workflow when one is detected. Currently supported for Ethereum and Solana.

Oracle feeds. External data triggers based on oracle price updates or threshold crossings. When a Pyth price feed crosses a defined boundary, the workflow fires. Useful for automated trading, liquidation monitoring, and alerts.

5.3 Step Kinds

Each step in a workflow invokes one of five step kinds.

Run (`run:`). Executes a shell command inside an isolated container. The command runs in a Docker container with a defined image, environment variables, and mounted volumes. Output is captured from stdout and parsed as step outputs. This is the most flexible step kind, suitable for custom logic, data transformation, and any operation not covered by a specialized action.

```yaml
- name: Process data
  run: |
    echo "Processing ${{ steps.query.outputs.count }} records"
    python3 transform.py --input data.json
```

Uses (`uses:`). Invokes a published action by reference. The action is identified by its namespace, name, and version. Inputs are passed via the `with:` block. Outputs are available to subsequent steps via `${{ steps.<id>.outputs.<key> }}`. This is how ecosystem partner capabilities are accessed.

```yaml
- name: Query blockchain data
  id: query
  uses: w3-io/w3-sxt-action@v1
  with:
    command: query
    sql: SELECT * FROM ethereum.transactions LIMIT 10
```

Ethereum (`ethereum:`). Native blockchain operations on Ethereum and EVM-compatible chains. Supports balance queries, token transfers and approvals, contract deployment and calls, event queries, NFT operations, and ENS name resolution. The `network` parameter selects the target chain.

```yaml
- name: Check balance
  ethereum:
    action: get-balance
    network: ethereum
    params:
      address: "0x742d35Cc6634C0532925a3b844Bc9e7595f2bD18"
```

Solana (`solana:`). Native blockchain operations on Solana. Supports balance queries, token account lookups, program invocation, SOL and SPL token transfers, and transaction monitoring.

Bitcoin (`bitcoin:`). Native blockchain operations on Bitcoin. Supports balance queries, UTXO management, fee estimation, and transaction construction.

W3 (`w3:`). Protocol-native operations where the workflow interacts with W3.io's own infrastructure. The first `w3:` operation is `prove-field`: the workflow generates a merkle inclusion proof for a specific field of its own consensus-attested step execution. This enables a compute-prove-settle pattern where a workflow executes a step, generates a cryptographic proof of a specific execution detail (which validator ran it, what the output was), and submits that proof to an L1 contract in a subsequent step.

```yaml
- name: Execute computation
  id: compute
  uses: w3-io/w3-hyperbolic-action@v1
  with:
    model: llama-3
    prompt: "Analyze this transaction"

- name: Prove who executed it
  id: proof
  w3:
    prove-field:
      index: 7  # runner field

- name: Submit proof on-chain
  ethereum:
    action: call-contract
    network: ethereum
    params:
      to: "0xVerifierContract"
      function: "verifyExecution(bytes32,bytes32,bytes32[])"
      args:
        - ${{ steps.proof.outputs.root }}
        - ${{ steps.proof.outputs.leaf }}
        - ${{ steps.proof.outputs.branch }}
```

The field-level proof composes with W3.io's settlement proofs. The settlement layer proves that a step run is included in a settled epoch (epoch root to step-run root via SMT proof). The field proof goes one level deeper, proving a specific field value within that step run. Together they form a complete proof path from an on-chain epoch root to an individual piece of execution data.

5.4 Expressions and Data Flow

W3.io's expression engine evaluates dynamic values at runtime using the `${{ }}` syntax. Expressions can reference:

- **Step outputs**: `${{ steps.query.outputs.result }}` accesses a named output from a previous step
- **Inputs**: `${{ inputs.token_address }}` accesses a trigger input parameter

- **Environment**: `${{ env.API_KEY }}` accesses an environment variable
- **Secrets**: `${{ secrets.PRIVATE_KEY }}` accesses a secret value (redacted in logs)
- **Job context**: `${{ job.status }}` reflects the current job state

The expression language supports functions for string manipulation (`contains`, `startsWith`, `endsWith`, `format`), data conversion (`toJSON`, `fromJSON`), and status checks (`success()`, `failure()`, `always()`).

Data flows between steps through outputs. Each step can produce named outputs that subsequent steps reference by step ID. This creates a typed data pipeline where each step receives exactly the data it needs from previous steps, without shared mutable state.

5.5 Conditionals and Failure Handling

Steps can be conditionally executed using the `if:` field.

```yaml
- name: Alert on failure
  if: failure()
  uses: w3-io/w3-email-action@v1
  with:
    to: ops@example.com
    subject: Workflow failed
```

The `continue-on-error` field controls whether a step failure stops the workflow or allows subsequent steps to execute. Combined with status check functions (`success()`, `failure()`, `always()`), this enables error handling patterns like cleanup operations, fallback logic, and notification on failure.

Each step has a configurable timeout via `timeout-minutes`. If a step exceeds its timeout, the execution backend terminates it and the step is marked as failed. The default timeout is 10 minutes. The effective timeout for any step is the minimum of the step's configured timeout and the remaining time in the job's overall timeout.

5.6 Compilation and Deployment

Workflow YAML is not interpreted directly by validators. It goes through a compilation pipeline.

Parse: the YAML is parsed into an abstract syntax tree. Structural errors (invalid keys, missing required fields, malformed expressions) are caught here.

Validate: semantic validation checks type consistency, expression references, step ID uniqueness, and action compatibility. Invalid expression references (referencing a step that doesn't exist, or an output that an action doesn't produce) are caught before deployment.

Compile: the validated AST is compiled into a canonical form. Comments, formatting, and whitespace are stripped. The result is a deterministic representation of the workflow's semantic content.

Hash: the compiled form is hashed. This `definition_hash` is the workflow's identity for settlement purposes. Two workflows with identical semantics but different formatting produce the same hash. The hash is stored on-chain in the WorkflowRegistry contract, creating an immutable link between the deployed workflow and its on-chain provenance.

Deploy: the compiled workflow is distributed to the validator network. Validators store the definition and begin monitoring for the specified trigger events.

6 Execution

6.1 The Separation of Concerns

W3.io's execution model separates **what** to execute from **how** to execute it. The workflow runtime decides what happens at each step: which action to invoke, what inputs to pass, how to parse the outputs. The execution backend decides how it happens: which container to run, how to enforce timeouts, how to handle failures.

This separation is implemented through two traits in the codebase.

The **Syscalls** trait defines five operations that the runtime can invoke: `run` (shell commands), `uses` (published actions), `ethereum`, `solana`, and `bitcoin` (native chain operations). The runtime calls these without knowing anything about the underlying execution infrastructure.

The **Backend** trait defines two operations that the syscalls layer delegates to: `prepare` (make the execution environment ready) and `execute` (run a command and return the result). The backend handles isolation, timeouts, resource management, and cleanup.

Today, W3.io ships with a Docker backend. The architecture is designed so that a Firecracker backend (microVM isolation) or a WASM backend can be added without changing the runtime or the syscalls layer.

6.2 Docker Execution

When a validator is selected to execute a workflow step, the Docker backend handles the full lifecycle.

Preparation. The backend pulls the required container image (if not already cached) and prepares the execution environment. Image pulls are deduplicated across concurrent workflows. If two workflows need the same image at the same time, only one pull happens.

Container naming. Each container gets a deterministic name based on the workflow's trigger hash, job ID, step index, and a timestamp. This makes containers identifiable in Docker logs, and it means the backend can kill a specific container by name if the process becomes unresponsive.

Execution. The step's command runs inside the container with the workflow's environment variables, secrets (injected via a temporary env file that is deleted after startup), and any required filesystem mounts. Standard output and standard error are captured as the step's log output.

Timeout enforcement. Every step has a timeout (configurable via `timeout-minutes:`, default 10 minutes). The backend uses `tokio::select!` with fair polling to race the step execution against a deadline. If the deadline fires first, the backend initiates a shutdown sequence:

1. Send SIGTERM to the Docker process (Docker forwards this to the container's PID 1)
2. Drain stdout and stderr during a 10-second grace period (prevents pipe deadlock where a process writing during shutdown blocks on full pipes)
3. If the process hasn't exited after the grace period, send SIGKILL
4. Unconditionally call `docker kill` by container name as a final defense

This escalation ensures that a hung container never blocks consensus indefinitely. The step is marked as failed with a timeout error, and the workflow's failure handling logic (continue-on-error, if conditions) determines what happens next.

Output parsing. After execution completes, the backend parses the container's output. Step outputs (key-value pairs written to a designated output file inside the container) are extracted and made available to subsequent steps via the expression engine. Environment modifications (variables written to the environment file) are merged into the workflow's environment for downstream steps.

6.3 Step Identity

Every execution carries typed identity fields that flow from the consensus layer through the entire call chain: the trigger hash (which workflow run), the job ID (which job within the run), and the step index (which step within the job). These fields appear in every backend log line, making it possible to trace a specific step's execution across container logs, P2P messages, and consensus records using the same identifiers.

This is not just a debugging convenience. The step identity is part of the settlement receipt. The trigger hash, combined with the step index, uniquely identifies a specific step execution within the protocol's history. When a receipt is verified against an on-chain epoch root, the step identity ties the cryptographic proof to a specific observable event.

6.4 Future Backends

The Backend trait is designed to support execution environments beyond Docker.

Firecracker. Firecracker microVMs provide stronger isolation than containers. Each step would execute in its own lightweight VM, booted in milliseconds, with a dedicated kernel. The `prepare` method would handle VM boot and filesystem setup. The `execute` method would run the command inside the VM. This is the planned path for production workloads where stronger isolation is required.

WASM. WebAssembly runtimes offer sandboxed execution without the overhead of containers or VMs. A WASM backend would compile actions to WASM modules and execute them in a sandboxed runtime. This is a longer-term consideration for lightweight actions where container startup overhead is disproportionate to the computation.

Both future backends would implement the same `prepare` and `execute` interface. The runtime, the consensus layer, and the settlement layer would not change. Only the isolation boundary moves.

7 Consensus

7.1 BOSCO

W3.io uses a single Byzantine fault-tolerant consensus algorithm for all protocol-level agreement. The protocol's consensus is inspired by BOSCO (Song and Renesse 2008) (which demonstrated that BFT agreement can complete in a single communication step under favorable conditions) and builds on the foundational work of practical BFT (Castro and Liskov 1999). W3.io's implementation extends these ideas into a leader-based, view-change-capable protocol optimized for the per-step consensus pattern of workflow execution.

The protocol tolerates up to f faulty or adversarial validators in a committee of 3f+1 members. In a 10-member committee, up to 3 members can be Byzantine (offline, malicious, or sending conflicting messages) and the protocol still reaches correct consensus.

The consensus protocol operates in three phases.

Propose. The committee leader constructs a proposal and broadcasts it to all committee members. The proposal contains the value being decided on (which validator should run a step, what the attestation set looks like, which receipts to include in an epoch) along with the leader's identity and a round number. The leader is selected deterministically from the committee using the round number, so all honest members agree on who the current leader is.

Vote. Each committee member receives the proposal, validates it against their local state (does the proposed runner exist in the validator set? does the receipt hash match what they received via

gossipsub?), and broadcasts a vote. A vote either confirms the proposal or rejects it. Votes are signed and include the round number to prevent cross-round replay. Each member collects votes from other members as they arrive over P2P.

Decide. When a member has collected 2f+1 matching votes for the same proposal in the same round, the decision is final. The member applies the decision locally (advance the state machine, record the attestation, close the epoch). The 2f+1 threshold guarantees that any two quorums overlap by at least one honest member, which means two conflicting decisions in the same round are impossible.

View change. If the leader fails to propose within a configurable timeout, any member can initiate a view change. The round number increments, a new leader is selected deterministically, and the protocol restarts from the Propose phase with the new leader. This is automatic. No operator intervention is needed. A faulty leader costs one timeout period (typically seconds), not a stalled workflow.

View changes are critical for liveness. Without them, a single offline or malicious leader could block all workflow execution indefinitely. With them, the worst case is a brief delay while leadership rotates. The protocol cycles through leaders until it finds one that responds.

The consensus protocol is not a novel construction. It follows the established BFT pattern found in production systems across the blockchain industry (Castro and Liskov 1999), extended with ideas from BOSCO's one-step optimization for the common case where no faults or contention are present (Song and Renesse 2008). W3.io's implementation is 2,720 lines of Rust (Matsakis and Klock 2014), tested under multi-node deployments with injected Byzantine faults including equivocating leaders, message delays, and partitioned networks.

W3.io chose a single consensus algorithm rather than offering pluggable consensus per workflow. This is deliberate. Pluggable consensus adds complexity without proportional benefit. The security properties of the network depend on one well-tested algorithm, not a menu of options with varying guarantees. The same consensus protocol is used everywhere agreement is needed. The contexts differ, but the mechanism is the same.

7.2 The Workflow State Machine

When a workflow is triggered, W3.io's consensus engine drives it through a four-state machine. Each state represents a phase of the workflow step lifecycle, and each transition requires committee agreement.

State 1: Awaiting Trigger. A trigger event arrives: a cron tick, an RPC call, a chain event, or an oracle update. The protocol must first confirm that the trigger is legitimate. For RPC triggers, this means verifying the caller's signature against the namespace's authorization policy. For chain

events, this means confirming the event actually occurred on-chain through a monitor group (a small committee of validators assigned to watch that specific event type).

Once the trigger is confirmed, a workflow committee is selected from the active validator set using the deterministic shuffle described below. The committee members receive the trigger evidence and the workflow definition. The run begins.

State 2: Awaiting Step Runner Proposals. For each step in the workflow, the committee must agree on which validator executes it. The committee leader examines the eligible validators (those online, with sufficient capability, and not already overloaded) and proposes a runner. Committee members verify that the proposed runner is eligible and vote to confirm.

If the leader fails to propose within the round timeout, view change rotates leadership. If the selected runner accepts but then fails to return a result within the step timeout, the committee marks the runner as failed and the leader proposes a new runner. After M consecutive runner failures for the same step (configurable, default 3), the workflow itself is marked as failed. This prevents infinite retry loops on steps that are fundamentally broken.

State 3: Awaiting Step Execution. The selected runner executes the step using the execution backend (Execution). This is the only phase where actual computation happens. The runner pulls the container image (if needed), injects the step's inputs and environment, runs the command, and captures the outputs.

When execution completes, the runner broadcasts the result to the committee: the step's outputs, the execution duration, the exit code, and a hash of the produced workflow block. The workflow block contains the step's inputs, outputs, the runner's identity, and the previous block's hash, forming a per-run hashchain.

State 4: Awaiting Step Attestation. Committee members independently assess the runner's result. Each member produces an attestation with one of three values:

- **Confirmed**: the result appears correct based on the member's local validation
- **Rejected**: the result is inconsistent with what the member expected (for example, the output hash doesn't match a deterministic recomputation)
- **Indeterminate**: the member cannot determine correctness (for example, the step interacted with an external API that the member cannot independently verify)

The committee runs BOSCO to agree on the canonical attestation set. A deterministic decision rule produces the final verdict from the agreed assessments. If the majority confirmed, the step passes and the state machine advances to State 2 for the next step. If the majority rejected, the step fails and the workflow's failure handling logic determines whether execution continues.

When all steps complete, the workflow produces a receipt containing the trigger hash, the final

state (completed or failed), timing data, the committee seed, the definition hash, the namespace, and the hashes of all consensus blocks. This receipt enters the settlement pipeline described in the Settlement section.

7.3 Committee Selection

Committee selection determines which validators participate in consensus for a given workflow execution. The selection must be deterministic (all honest nodes compute the same committee), unpredictable (no party can influence or predict committee composition before the trigger), and capability-aware (only validators with the required capabilities are eligible).

W3.io achieves this with a seeded Fisher-Yates shuffle.

The seed is derived from two values: the cumulative settlement root and the trigger hash. The cumulative root is the global sparse merkle tree root from the most recently settled epoch. It depends on every workflow receipt ever settled across all namespaces and all prior epochs. An attacker trying to predict or influence committee membership would need to control the settlement of all workflows across the entire network, not just the target workflow. The trigger hash adds per-run entropy that is unique to this specific workflow execution. Together they function like a verifiable random function output: deterministic, unpredictable before the epoch settles, and publicly verifiable by anyone who knows the on-chain root.

The seed feeds a ChaCha20 pseudorandom number generator (Nir and Langley 2018), which drives the Fisher-Yates shuffle over the eligible validator set. ChaCha20 is a stream cipher used here as a cryptographically secure PRNG. It replaced an earlier linear congruential generator that was predictable and trivially invertible. The security of committee selection depends on the PRNG being non-invertible. ChaCha20 provides this.

Before shuffling, the validator set is filtered by capabilities. Each validator declares a capability bitmap at registration: whether it maintains an Ethereum WebSocket connection, an Avalanche WebSocket connection, GPU compute access, or other resources. A workflow that triggers on Ethereum events requires validators with the Ethereum WebSocket capability. The shuffle operates only on validators whose capabilities satisfy the workflow's requirements, ensuring that selected committee members can actually perform the work.

The minimum committee size in production is configurable. Larger committees provide stronger Byzantine fault tolerance (more members must be compromised) but increase the P2P message overhead per step. The default minimum is 10, giving f=3 tolerance.

7.4 Four Consensus Contexts

BOSCO is used in four distinct contexts within the protocol. The algorithm is identical in each case. What differs is the proposal value, the committee composition, and the committee lifetime.

Step runner selection. For each workflow step, the per-run committee agrees on which validator executes it. The proposal value is the selected runner's identity. This is the highest-stakes use of BOSCO because it directly determines who controls the step's execution. Committee size follows the configured minimum (default 10 members, f=3). The committee is formed at trigger time and persists for the duration of the workflow run.

Step attestation. After execution, the same per-run committee agrees on the canonical assessment of the result. The proposal value is the attestation set (each member's confirmed/rejected/indeterminate verdict). This separates execution from evaluation: the runner produces the result, but the committee decides whether to accept it. A single compromised runner cannot force a bad result through consensus because the committee independently evaluates the output.

Trigger confirmation. Dedicated monitor groups run BOSCO to confirm external events before dispatching workflow execution. A monitor group is a small committee (7 members, f=2) assigned to a specific trigger type. Multiple workflows that watch the same event (for example, the same ERC-20 Transfer event on the same contract) share a single monitor group. This prevents trigger spoofing: a single validator claiming an event occurred is not sufficient to start a workflow. The monitor group must reach BFT agreement on the trigger evidence, including temporal validation that rejects future-dated events.

Epoch manifest agreement. The aggregation committee uses BOSCO to agree on which workflow receipts are included in each settlement epoch. The aggregation committee is larger (30 members, f=9) and longer-lived (persists for an entire epoch term, not just one workflow run). The proposal value is the receipt manifest: a sorted list of receipt hashes. Once the manifest is agreed upon, every honest member independently builds the same sparse merkle tree from the same receipts, producing the same cumulative root. This root is then signed and submitted to the L1 contract (Settlement).

In every context, the same properties hold: 2f+1 agreement is required, leader failure triggers automatic view change, and the decision is deterministic given the same inputs. W3.io does not need four consensus algorithms. It needs one good one, applied consistently.

8 Settlement

8.1 Why Settlement Matters

W3.io workflows execute off-chain. Validators run containers, reach consensus on results, and produce outputs. But without settlement, verifiability ends at the validator set. A third party who was not part of the consensus has no way to independently confirm that a workflow ran, what it produced, or when it executed.

Settlement bridges this gap. It takes the outputs of off-chain consensus and anchors them to a public blockchain, creating a permanent record that anyone can check. After settlement, anyone with access to the L1 can confirm that a BFT quorum attested to a specific workflow result. No access to the validator network required. No trust in any individual validator.

This is the difference between "the validators say it happened" and "the chain proves the validators agreed it happened." The settlement layer proves the attestation occurred and was signed by a quorum. It does not independently re-execute the computation (see Risk Factors, the Risk Factors section, for the distinction between attested execution and computational integrity proofs).

8.2 Receipts

When a workflow run completes, the consensus engine produces a **receipt**. The receipt is a compact summary of everything that matters about the execution. It contains 15 fields:

- **version**: receipt format version (currently 1)
- **namespace**: the namespace that owns the workflow
- **workflow_name_hash**: keccak256 hash of the workflow name
- **trigger_hash**: unique identifier for this specific run
- **definition_hash**: hash of the compiled workflow definition, matching the on-chain registry
- **l1_anchor_hash**: the L1 block hash at the time the workflow was triggered
- **final_block_hash**: hash of the last consensus block in the run's hashchain
- **end_state**: completed or failed
- **failure_reason**: if failed, why (runner timeout, attestation rejection, step error)
- **failure_data_hash**: hash of the failure details for diagnostic retrieval
- **actions_used**: sorted list of namespace-scoped action ID hashes used in the run
- **step_count**: total number of steps executed
- **start_time**: when the workflow started (UTC milliseconds)
- **end_time**: when it completed
- **committee_seed**: the seed used for committee selection, enabling anyone to reconstruct which validators participated

The receipt is ABI-encoded using the same layout as Solidity's `abi.encode`, producing a byte-for-byte match between the Rust and Solidity implementations. The receipt hash is `keccak256(abi.encode(receipt))`. This hash is what gets inserted into the sparse merkle tree and ultimately anchored on-chain.

The ABI encoding compatibility is verified by a golden test: the Rust implementation's output is compared against the output of `cast abi-encode` (the Solidity reference encoder) for identical inputs. If the encoding ever drifts, the test fails before the code ships.

8.3 Epochs

Receipts are not settled individually. They are batched into **epochs**. An epoch is a fixed-duration window during which receipts accumulate. When the epoch closes, the aggregation committee agrees on the receipt manifest (the list of receipt hashes to include), builds the merkle tree, and submits the commitment to the L1.

Epoch mechanics:

- Epochs are numbered sequentially. Empty epochs (no workflows ran) are skipped. The contract accepts the next non-empty epoch regardless of how many empty epochs elapsed.
- Each epoch references the previous epoch's cumulative root, creating a hashchain of epochs on the L1. This prevents replay, reordering, and forking.
- A receipt that arrives after the epoch closes goes into the next epoch. The cutoff is hard. There is no grace period. Late receipts are not lost, they are deferred.
- The receipt manifest is agreed upon by the aggregation committee using BFT consensus (Consensus). This ensures all honest validators build the tree from the same receipt set.

Batching into epochs serves two purposes. First, it amortizes the gas cost of L1 submission across many workflow executions. At 100 workflows per epoch, the per-workflow settlement cost approaches zero. Second, it produces a cumulative root that covers the full history of all settled workflows, not just the current epoch. This is what makes historical verification possible without scanning the entire chain.

8.4 The Two-Level Sparse Merkle Tree

W3.io uses a two-level cumulative sparse merkle tree (Gao et al. 2021) to organize settled receipts.

Level 1: Global tree. The leaves are namespace roots, keyed by `keccak256("ns" || namespace_id)`. Each leaf's value is the root of that namespace's own tree. The global tree is sparse: only namespaces with at least one settled receipt have leaves. A namespace created on-chain but never used does not appear in the tree.

Level 2: Per-namespace trees. Each namespace has its own tree. The leaves are receipt hashes, keyed by `keccak256(version || namespace_id || trigger_hash || workflow_name_hash)`. Each leaf's value is the receipt hash.

Both trees use keccak256 hashing for EVM compatibility. The tree implementation wraps the Jellyfish Merkle Tree library (originally developed for the Diem/Aptos blockchain), which provides efficient path compression, incremental updates, and both inclusion and non-inclusion proofs.

The cumulative root after processing epoch N is the on-chain root for epoch N. It reflects the entire history of all receipts across all namespaces, not just epoch N's receipts. Like Ethereum's state trie, each epoch's root is a snapshot of the full world state at that point.

This structure enables three types of verification:

Inclusion proof. Prove that a specific workflow ran and settled. The verifier needs the receipt, a per-namespace merkle proof (receipt exists in the namespace tree), and a global merkle proof (namespace root exists in the global tree). Two proofs, checked against the on-chain root. Typical proof size is 2-3 KB.

Non-inclusion proof. Prove that a workflow did NOT run. The sparse merkle tree supports non-membership proofs: given a key, prove that no leaf exists at that position. This is critical for dispute resolution and billing ("you claim we didn't run your workflow; here's cryptographic proof that we did" or "you claim you ran it; prove it against the root").

Composability. Other smart contracts can verify workflow results on-chain. The `SMTVerifier` contract exposes a `verifyWorkflowResult()` function that accepts a receipt, two proofs, and an epoch number, and returns whether the receipt is valid against the on-chain root. A DeFi protocol can gate actions on workflow completion: "release payment only if this compliance workflow settled successfully."

8.5 L1 Anchoring

The final step in settlement is submitting the epoch commitment to the L1 contract. This is a two-phase process.

Phase 1: Manifest agreement. The aggregation committee leader proposes the receipt manifest for the epoch. Committee members run BFT consensus to reach consensus on which receipts are included. Once agreed, every honest member independently builds the same sparse merkle tree from the same receipts (deterministic: same receipts in the same order produce the same root).

Phase 2: Root signing. After building the tree, committee members produce a quorum signature over the epoch tuple: chain ID, contract address, epoch number, validator set version, cumulative root, previous root, receipt count, and namespace count. The chain ID and contract address serve

as a domain separator, preventing signatures from one deployment being replayed on another. The quorum signature is an aggregate BLS12-381 signature (Boneh et al. 2022) verified on-chain via EIP-2537 precompiles (Vlasov and Olson 2020).

The `EpochSettlement` contract verifies:

- The epoch number is greater than the last submitted epoch (monotonically increasing)
- The previous root matches the stored root from the last epoch (chaining, no forks)
- The quorum signature is valid against the known validator set
- The validator set version matches the current on-chain version

If all checks pass, the contract stores the new cumulative root, emits an `EpochSubmitted` event, and the epoch is settled. The cumulative root is now publicly queryable by any contract or off-chain verifier.

Submission incentives. The manifest consensus leader has priority to submit during a configurable window (default 30 seconds). After the window expires, any validator can submit, incentivized by a gas reimbursement bounty. This ensures that a single faulty leader cannot block settlement. The contract's idempotent design means racing submitters are harmless: the first valid submission is accepted, duplicates revert cheaply.

8.6 Crash Recovery

Settlement is designed to survive node failures without losing data.

W3.io uses a write-ahead log (WAL) to track receipts and epoch state. Every receipt written to the WAL is durable (when backed by MDBX persistent storage). Every epoch status transition (open, closed, submitted, settled, failed) is recorded. If a node crashes mid-epoch, it restarts, opens the WAL (which self-heals to the last committed transaction), checks the L1 for the latest settled epoch, and resumes from where it left off.

The recovery sequence:

1. Open the WAL. MDBX guarantees consistency after unclean shutdown.
2. Read the latest settled epoch from the L1 contract. This is the source of truth.
3. Check the WAL for unsettled epochs. Any epoch in Closed or Submitted status needs re-processing or re-submission.
4. Backfill missing receipts from peers for the last 2 epochs.
5. Re-process if needed. Same receipts plus same manifest produces the same root. Deterministic replay is safe.
6. Check for in-flight L1 submissions. If an epoch was submitted but the WAL doesn't know, scan for it on-chain. The contract rejects duplicates.

The worst case after a power failure is a brief delay while the node catches up. No data is lost. No re-sync from genesis is required.

9 Cryptography

9.1 Building on Giants

W3.io does not invent new cryptographic primitives. Every algorithm used in the protocol is a well-established, audited, production-grade construction with years of deployment in other systems. The protocol's security properties are inherited from these foundations, not asserted by novel constructions.

This is a deliberate choice. Novel cryptography is interesting research but a poor foundation for a production protocol. The attack surface of a new primitive is unknown by definition. The attack surface of BLS12-381 is understood by thousands of researchers and audited by every major Ethereum consensus client team. W3.io benefits from that scrutiny without paying for it.

The following table summarizes the cryptographic primitives used in the protocol, their purpose, and their provenance.

Primitive	Purpose	Standard/Library	Used By
BLS12-381	Quorum signatures on epoch submissions, receipt signing, trigger confirmation, validator proof of possession	blst (Supranational 2023), draft-irtf-cfrg-bls-signature (Boneh et al. 2022)	Ethereum 2.0 (Lighthouse, Prysm, Teku, Lodestar, Nimbus), Cosmos, Polkadot
keccak256	Receipt hashing, namespace ID derivation, SMT key derivation, ABI encoding	Keccak (pre-NIST, Ethereum variant)	Ethereum (native hash function), Solidity `abi.encode`
ChaCha20	Deterministic committee selection PRNG	RFC 8439 (Nir and Langley 2018)	TLS 1.3, WireGuard, Linux kernel CSPRNG

Primitive	Purpose	Standard/Library	Used By
Ed25519	Validator identity keys, P2P message signing	RFC 8032	libp2p, Signal, SSH, Tor
SHA-256	Workflow run block hashchaining, trigger hash derivation	FIPS 180-4	Bitcoin, TLS, nearly every production system
Jellyfish Merkle Tree	Two-level cumulative sparse merkle tree for settlement state	Diem/Aptos (Gao et al. 2021)	Aptos, Penumbra
SSZ	Wire format for P2P message serialization	Ethereum SSZ specification (Foundation 2019)	Ethereum 2.0 consensus layer

9.2 BLS12-381

W3.io uses BLS12-381 signatures for all protocol-level aggregate signing. BLS (Boneh-Lynn-Shacham) signatures have a unique property that makes them ideal for quorum-based systems: multiple individual signatures can be aggregated into a single signature of constant size, and the aggregate can be verified against the aggregate public key in a single operation. A quorum of 20 validators produces a signature no larger than one validator's signature.

W3.io's BLS implementation wraps the `blst` library (Supranational 2023), the same library used by every major Ethereum 2.0 consensus client. The protocol uses the min-pk scheme (public keys in G1, signatures in G2), matching Ethereum 2.0's choice. On-chain verification uses EIP-2537 precompiles (Vlasov and Olson 2020) for gas-efficient pairing checks. W3.io operates as a custom Avalanche L1 where the chain binary is under the protocol's control, and EIP-2537 precompiles are included in the custom chain configuration.

Four domain separation tags (DSTs) prevent cross-context signature replay:

Context	DST	Purpose
Epoch quorum	`W3IO-BLS-EPOCH-V1`	Validators sign the epoch tuple for L1 submission

Context	DST	Purpose
Receipt gossip	`W3IO-BLS-RECEIPT-V1`	Committee members sign individual receipts
Trigger confirmation	`W3IO-BLS-TRIGGER-V1`	Monitor group members sign trigger evidence
Proof of possession	`W3IO-BLS-POP-V1`	Validators sign their own public key at registration

A signature produced under one DST cannot be accepted under another, even if the underlying message bytes are identical. This is enforced at the library level: the DST is a required parameter to every sign and verify call.

Every validator must submit a proof of possession (PoP) at registration. The validator signs their own compressed G1 public key using the `W3IO-BLS-POP-V1` DST. The on-chain `ValidatorSet` contract verifies this signature before accepting the key. This prevents rogue public key attacks, where an attacker crafts a public key that cancels out honest validators' keys during aggregation.

W3.io enforces PoP at the type level. The `PopVerifiedKey` type can only be constructed by passing PoP verification. The `verify_aggregate` function requires `PopVerifiedKey` inputs, not raw public keys. This makes it a compile-time error to use an unverified key in aggregate signature verification.

9.3 The Hash Boundary

W3.io uses two hash functions for different layers of the protocol, with a clean boundary between them.

keccak256 is used for everything that touches the settlement layer or needs EVM compatibility: receipt hashing (`keccak256(abi.encode(receipt))`), namespace ID derivation (`keccak256(creator, nonce)`), SMT key derivation, and any value that will be verified by a Solidity contract. Keccak256 is Ethereum's native hash function (Buterin 2014). Note that Ethereum's keccak256 uses the original Keccak submission, not the NIST-standardized SHA3-256 (FIPS 202), which uses different padding. The two produce different outputs for the same input. W3.io uses the Ethereum variant for settlement compatibility.

SHA-256 is used for protocol internals that never cross the EVM boundary: workflow run block hashchaining, trigger hash derivation, and monitor group seed computation. SHA-256 is used in 17 files across 8 crates in the protocol codebase.

The boundary is enforced through the type system. Settlement-layer types use `[u8; 32]` values produced by keccak256. Protocol-internal types use separate newtypes. A keccak256 hash cannot

be accidentally passed where a SHA-256 hash is expected.

9.4 ChaCha20 for Committee Selection

Committee selection requires a PRNG that is deterministic (all nodes compute the same shuffle), cryptographically secure (the output cannot be predicted or inverted), and efficient (committee selection happens frequently). ChaCha20 (Nir and Langley 2018) meets all three requirements.

The seed is a SHA-256 hash of the cumulative settlement root concatenated with the trigger hash. This seed initializes a ChaCha20 stream cipher, which produces the random bytes needed to drive a Fisher-Yates shuffle over the validator set. The same seed always produces the same committee. Different seeds produce unpredictable committees.

ChaCha20 replaced an earlier linear congruential generator (LCG) that was trivially invertible. An attacker who observed a committee could invert the LCG to recover the seed and predict future committees. ChaCha20 does not have this property. The committee selection change is consensus-breaking: all validators must upgrade simultaneously.

10 Namespaces

10.1 Meeting People Where They Are

Most blockchain protocols force a flat ownership model. One address owns one account. One account deploys one contract. Organizational structure, if it exists at all, is layered on top through multi-sig wallets or custom access control contracts. This works for individual developers. It does not work for companies, teams, partnerships, or any organization more complex than one person with one wallet.

Real organizations are hierarchical. A company has departments. Departments have teams. Teams have members. Some resources are shared. Some are isolated. Billing might be centralized or delegated. Authority might cascade from the top or be granted locally. The organizational structure changes over time: departments merge, teams spin out, companies get acquired.

W3.io's namespace model is designed to meet these organizations where they are. A solo developer creates a namespace and deploys workflows. A company creates a namespace with sub-namespaces for each department. A venture studio creates namespaces for each portfolio company under a parent namespace. An open-source project creates a namespace with no authority and no payer, and anyone can deploy to it, paying for their own runs.

The same primitive handles all of these cases. No rigid tiers, no enterprise-only features, no upgrade path from "personal" to "organization." Just a tree that grows with you.

10.2 Namespace Identity

Every namespace is identified by a permanent 32-byte value derived from its creator's address and a per-creator nonce:

```
namespace_id = keccak256(creator_address, nonce)
```

The creator is immutable, like a contract deployer. It is recorded on-chain but is not the ongoing authority. This is a critical distinction. The identity of a namespace never changes, regardless of who controls it, who pays for it, or where it sits in the hierarchy. Receipts, SMT keys, and on-chain references all use the namespace ID. If the ID changed when authority transferred, every historical reference would break.

The nonce allows a single creator address to produce multiple namespaces. The default nonce (all zeros) works for creators who only need one. A creator who needs many (staging, production, per-client) increments the nonce.

10.3 Three Roles

Each namespace has three identity fields that serve different purposes.

Creator is the address that created the namespace. Immutable. Recorded on-chain for provenance. Not an ongoing role. The creator has no special powers after creation, just as a contract deployer has no special powers over a deployed contract (unless the contract grants them).

Authority is the address that controls the namespace. The authority can deploy and undeploy workflows, set access policies, manage child namespaces, and initiate transfers. Authority is transferable through a two-step process with a cancellation window, protecting against instant namespace theft via a compromised key.

Payer is the address that pays for workflow executions in the namespace. Payer is transferable independently of authority. This separation is fundamental. The person who controls what workflows run is not necessarily the person who pays for them. A company's CTO controls the deployment pipeline. The company's treasury pays for the compute.

All three roles can be set at creation time or left empty to inherit from the parent.

10.4 Hierarchy and Inheritance

Namespaces form a tree rooted at a `w3` root namespace. Any namespace can be a parent. The tree can be arbitrarily deep.

```
w3 (root: authority=none, payer=none)
  acme (authority=0xCEO, payer=0xTreasury)
```

```
    engineering (authority=inherited, payer=0xEngBudget)
    finance (authority=0xCFO, payer=inherited)
  opensource-project (authority=none, payer=none)
```

Authority and payer inherit down the tree by default. If a namespace has no explicit authority, the protocol walks up the tree until it finds one. If a namespace has no explicit payer, the same walk happens. The root namespace has no authority and no payer. Inheriting all the way to root means "open": anyone can act as authority (public namespace), and each caller pays for their own workflow runs.

This inheritance model makes common patterns simple.

A **company** sets authority and payer on its top-level namespace. Every child namespace inherits both unless explicitly overridden. The engineering team gets its own budget (overrides payer) but inherits the CEO's authority for deployment control.

An **open-source project** sits under root with no authority and no payer. Anyone can deploy workflows. Anyone can trigger them. Each user pays for the runs they initiate. No gatekeeper.

A **multi-entity partnership** creates a shared parent namespace with joint authority (a multi-sig), and each partner company has a child namespace with its own authority and payer. Shared infrastructure lives in the parent. Partner-specific workflows live in the children.

A parent's authority can intervene in children: freeze operations, propose authority changes. But local authority takes precedence for day-to-day operations. The parent's ability to intervene is an administrative backstop, not day-to-day control.

10.5 Reparenting

Namespaces can move. A child can transfer from one parent to another, or move to root to become a top-level namespace. This models real organizational changes: spinouts, acquisitions, reorganizations.

Reparenting requires consent from both sides. The child's authority agrees to leave the current parent. The new parent's authority agrees to accept the child. The old parent does not get a veto. The child is sovereign enough to leave.

This models:

- **Spinout**: a department becomes its own company. The child namespace moves to root. Root is open, so no consent is needed from root.
- **Acquisition**: company B acquires a division of company A. The child namespace moves from A's tree to B's tree. Both B's authority and the child's authority must consent.
- **Reorganization**: a team moves from one department to another within the same company.

The child namespace moves between sibling namespaces.

Reparenting uses the same two-step process with cancellation window as authority transfer. The namespace ID does not change when reparented. All historical receipts, SMT entries, and on-chain references remain valid.

10.6 Namespace-Scoped Actions

Actions (the protocol-level representation of ecosystem partner capabilities) are scoped to namespaces. Each action's identity is derived from its owning namespace and action name:

```
action_id = keccak256(namespace_id || action_name)
```

This means the same action name in two different namespaces produces two different action IDs. A partner's `compliance-check` action in the `chainalysis` namespace is a different protocol object from a `compliance-check` action in a different namespace. There is no global action name collision.

Workflow receipts include the `actions_used` field: a sorted list of action IDs for every action invoked during the run. This creates a per-execution record of which partner capabilities were consumed, enabling per-partner fee attribution, billing, and analytics.

W3.io reserves a protocol namespace for built-in actions (HTTP, blockchain primitives, core utilities). Only the protocol can register actions in this namespace. Partner namespaces are self-governed by their authority.

10.7 Path to Chain-Native Namespaces

W3.io operates as an Avalanche L1, which means the chain binary is under the protocol's control. This opens an architectural path that pure EVM protocols do not have: implementing namespaces as a chain-native precompile rather than a Solidity contract.

A precompile executes at native speed, not EVM bytecode interpretation speed. Namespace operations like resolving the authority or payer for a given namespace ID require walking up the inheritance tree, potentially multiple storage reads at each level. As a Solidity contract, this is functional but expensive. As a precompile, it becomes a single native call that returns in microseconds.

More importantly, a precompile makes namespaces a first-class primitive of the chain itself. Other contracts on the L1 resolve namespace authority as cheaply as they call `ecrecover`. The settlement contracts, the workflow registry, and any future protocol contracts can authorize operations through the namespace precompile without the overhead of cross-contract calls.

Namespaces are initially deployed as a Solidity contract with a UUPS upgrade path. Once the interface is finalized and battle-tested in production, the namespace logic migrates to a precom-

pile. The Solidity contract becomes a thin wrapper that delegates to the precompile, maintaining backward compatibility for any contracts that reference the original address. This staged approach avoids baking an unproven interface into the chain binary while keeping the long-term architecture clean.

If the namespace model proves its utility in production, the interface is a candidate for standardization as an EIP or AIP, enabling other EVM protocols to adopt hierarchical namespace resolution as a shared primitive.

11 Validators

11.1 Becoming a Validator

W3.io's validator set is permissioned. Every validator operator is KYC-verified before joining the network. This is consistent with the protocol's focus on enterprise financial workflows, where the identity and accountability of consensus participants matters.

To join the network, an operator:

1. Completes KYC/KYB verification through the Foundation's approved provider
2. Posts W3 token collateral to the `ValidatorSet` smart contract
3. Submits a BLS12-381 proof of possession (signs their own public key with the `W3IO-BLS-POP-V1` DST)
4. Declares a capability bitmap describing what the validator can do
5. Enters the join queue

The on-chain contract verifies the proof of possession before accepting the key. This prevents rogue public key attacks on aggregate signatures. Once verified, the validator enters a sync period where it downloads recent state from peers. After sync completes, the validator becomes active and is eligible for committee selection at the next epoch boundary.

Epoch-boundary activation ensures that all validators agree on the validator set composition at any given time. A validator that registers mid-epoch does not participate in consensus until the next epoch starts. This prevents disagreements about committee membership during active consensus rounds.

11.2 Capability Bitmaps

Not all validators are equal. Some maintain Ethereum WebSocket connections for chain event monitoring. Some have Avalanche WebSocket connections. Some have GPU compute resources. The protocol needs to select committees that can actually perform the required work.

Each validator declares a capability bitmap at registration: a 64-bit value where each bit position represents a specific capability.

Bit	Capability	Meaning
0	`CORE_PROTOCOL`	Can run basic workflow steps
1	`ETHEREUM_WS`	Maintains Ethereum WebSocket connection
2	`AVALANCHE_WS`	Maintains Avalanche WebSocket connection
3-63	Reserved	Future protocol-defined capabilities

When the committee selection algorithm runs for a workflow that monitors Ethereum events, it filters the validator set to only those with the `ETHEREUM_WS` bit set before shuffling. A validator without the required capability is never selected for work it cannot perform.

Capabilities are self-declared in v1. The validator claims what it can do, and the protocol trusts the claim. A future upgrade (tracked as a post-v1 fork point) adds capability proofs: a validator proves it has an Ethereum WebSocket by signing a recent Ethereum block hash, demonstrating live access to the chain.

11.3 Join and Leave Lifecycle

Validators have a lifecycle managed by the protocol's lifecycle manager.

Joining. A new validator moves through: registration (on-chain), sync (download state from peers), activation (eligible for selection). The sync period prevents a new validator from being selected before it has the context needed to participate in consensus.

Leaving voluntarily. A validator signals intent to leave. It is immediately removed from future committee selections. However, it may still be part of in-flight workflow runs. The protocol drains active work: the validator completes any runs it is currently participating in, then fully exits. This prevents a departing validator from breaking consensus mid-workflow.

Force-exit. If a validator becomes unresponsive, the heartbeat mechanism detects it and initiates a force-exit. The validator enters the leave queue as if it had signaled voluntarily, but with a forfeiture penalty on its staked collateral (see Token Utility).

Rejoining. A validator that left can rejoin by going through the registration and sync process again. The force-exit counter is permanent per validator identity (Ed25519 key). A validator that force-exits repeatedly faces escalating penalties.

11.4 Heartbeat and Liveness

Validators publish a heartbeat message every 30 seconds on the `validator-heartbeat` gossipsub topic. Each heartbeat contains the validator's public key, a timestamp, and the latest epoch number the validator has processed.

Other validators track incoming heartbeats. If a validator misses 3 consecutive heartbeats (90 seconds of silence), the tracking validator initiates a force-exit through the lifecycle manager.

The heartbeat mechanism includes a grace period for new validators. When a validator is first discovered (either because it just joined or because the tracking node just started), it receives a synthetic heartbeat at the current time. This gives the new validator a full 90-second window to send its first real heartbeat before any force-exit logic triggers. Without this grace period, a coordinated upgrade (where all validators restart simultaneously) would cause every validator to immediately force-exit every other validator.

The heartbeat is the protocol's liveness detector. It does not assess whether a validator is performing well, only whether it is alive. Performance assessment happens through the consensus mechanism itself: a validator that consistently fails to execute steps or produce valid attestations will be re-selected around, reducing its effective participation without requiring an explicit penalty beyond the consensus level.

11.5 Progressive Decentralization

W3.io is designed to progressively decentralize control over the validator set. At launch, the Foundation operates the KYC allowlist and holds administrative keys through a team multi-sig with timelock controls. Over time, the protocol transitions toward a model where validator set participation is determined by on-chain staking mechanics: meet the minimum stake, pass KYC, submit proof of possession, join the set. The contract architecture supports this transition through transferable role administration.

12 Ecosystem

12.1 The B2B2C Model

W3.io is not a developer tool that sells directly to end users. It is infrastructure that powers ecosystem partners, who in turn serve their own communities and customers. This B2B2C model (business to business to consumer) is how W3.io generates network effects without requiring W3.io itself to acquire end users.

Approximately 20 network partners will operate at launch. Each partner operates a **subnet**: a staked position in the network that grants access to W3.io's execution and settlement infrastruc-

ture. Partners use this access to build solutions for their own markets, whether that is payments, compliance, data analytics, AI, or decentralized finance.

Partners are not passive integrations. Each partner stakes W3 tokens to operate their subnet, is required to distribute a portion of their allocated tokens to their community participants, and operates subnet infrastructure to deliver services to their community. This creates aligned incentives: partners succeed when they drive real usage through the network, not when they hold tokens passively.

The community participants within each partner's subnet take on active roles:

- **Sales teams** drive adoption by onboarding end users, driving measurable adoption outcomes
- **Solution builders** develop full-scale applications on the network, building applications that generate usage on the network
- **Validators** (also referred to as guardian operators) maintain the integrity and security of the network, performing consensus participation, step execution, and trigger monitoring (see Section 9)

12.2 Named Partners

W3.io's ecosystem includes partners across the full spectrum of Web3 infrastructure.

Partner	Category	Capability
Space and Time	Data	ZK-proven SQL queries across EVM chains
Hyperbolic	AI/Compute	Decentralized AI inference
Pyth	Oracles	Real-time price feeds across 400+ assets
Chainalysis	Security	Sanctions screening and risk scoring
EigenLayer	Infrastructure	Restaking and shared security
Dentity	Identity	Decentralized identity verification
Kite AI	AI	AI model serving and training
Yelay	DeFi	Yield optimization infrastructure
Circle	Payments	USDC stablecoin issuance and APIs
Stripe	Payments	Fiat-to-crypto payment rails

Each partner's capabilities are exposed as **actions** within the W3.io protocol: versioned, namespace-scoped, independently deployable components that any workflow can invoke. When a developer writes `uses: w3-io/w3-sxt-action@v1` in a workflow YAML, they are invoking Space and Time's query capability through W3.io's action registry.

12.3 The Action Registry

Actions are the protocol's unit of capability. The action registry tracks every published action, its owning namespace, its version history, and its interface specification.

Actions are published as container images with a standard entry point. The W3.io runtime handles image pulling, environment setup, input injection, output parsing, and timeout enforcement (Execution). Action authors only need to write the logic. The protocol handles the infrastructure.

The registry supports versioning. A workflow that references `w3-io/w3-sxt-action@v1` will always get the v1 interface, even after v2 is published. This prevents upstream action updates from breaking deployed workflows.

Receipts record which actions were used in each workflow execution through the `actions_used` field. This creates a complete audit trail of partner capability consumption, enabling per-partner attribution for fee attribution and analytics.

12.4 W3.cloud

W3.cloud is W3.io's decentralized compute and storage offering, providing AWS S3-compatible storage and container-based compute across a distributed infrastructure. W3.cloud serves as the infrastructure layer for workloads that are always-on (databases, API servers, streaming consumers) rather than event-triggered (workflows).

W3.cloud complements W3.io's workflow execution. Workflows handle event-driven, step-by-step computation with consensus on each step. W3.cloud handles persistent services that workflows interact with: databases that store workflow results, API servers that expose data to external consumers, and long-running processes that feed events into workflow triggers.

13 Token Utility

13.1 Four Participation Roles

The W3 token gates participation in the network. Every participant role requires staking tokens as collateral. This is not passive staking for yield. It is a commitment mechanism that aligns incentives with active contribution.

Ecosystem partners post collateral to operate a subnet. The subnet is the partner's position in the network: it grants access to W3.io's execution and settlement infrastructure, allows the partner to publish actions, and obligates the partner to distribute a portion of tokens to their community participants. Unstaking forfeits the remainder of the partner's allocation. This creates a strong disincentive to churn.

Solution builders post collateral to deploy applications on the network. Builders deploy applications on the network. The collateral requirement ensures that deployed applications have a committed maintainer.

Sales teams post collateral to participate in adoption programs. Sales participants drive measurable outcomes: users onboarded, workflows deployed, transaction volume generated. The collateral aligns sales incentives with long-term network health rather than short-term user acquisition.

Validators bond tokens to participate in consensus. Validator collateral is subject to slashing for provable misbehavior (equivocation, invalid signatures) and tiered forfeiture for repeated force-exits due to liveness failures. Validators participate actively: signing epochs, executing steps, monitoring triggers.

13.2 Fee Structure

Revenue from workflow execution flows in stablecoins (USDC), not in the W3 token. This separation is fundamental to the economic design. Businesses need predictable costs. A workflow that costs $0.05 today should cost approximately $0.05 tomorrow, regardless of token price movements. Stablecoin-denominated fees provide this predictability.

The W3 token is used for staking and access control. It is not the medium of exchange for workflow fees.

13.3 Staking Mechanics

Staked tokens are locked for the duration of the participant's active role. The lock is not time-based (stake for N months). It is role-based (stake while you are a partner, builder, sales participant, or validator).

If a participant decides to exit, the consequences depend on the role:

- **Partners**: unstaking forfeits the remainder of the initial allocation. Forfeited tokens are redistributed to active participants as part of the network's collateral maintenance process.
- **Validators**: voluntary exit follows the drain process (complete in-flight work, then fully exit). Collateral is returned minus any slashing penalties. Force-exit due to liveness failure triggers tiered forfeiture: first offense forgiven (hardware failures happen), escalating with repeated force-exits.
- **Builders and sales**: collateral returned on clean exit after any active commitments are fulfilled.

13.4 What the Token Is Not

The W3 token does not represent equity, debt, or ownership rights in W3.io or any affiliated legal entity. It does not confer rights to protocol revenues, dividends, or other distributions. Holding the token does not entitle the holder to any payment or consideration beyond the protocol utilities described above.

The token's design is focused on network participation and security. Active participation roles require token collateral. Holding the token without performing an active role does not generate any benefit.

14 Token Economics

Token economic parameters (total supply, allocation percentages, vesting schedules, and emission rates) are being finalized in coordination with legal counsel and are subject to change before the Token Generation Event. Specific values will be published when finalized.

14.1 Supply and Allocation

The total supply of W3 tokens will be fixed at genesis. There is no mechanism for minting additional tokens after the Token Generation Event. Allocation categories, percentages, and vesting schedules will be disclosed publicly before TGE.

14.2 Dual-Token Model

W3.io uses two currencies with distinct roles.

W3 token is used for staking, collateral, and access control. Participants post W3 tokens as collateral to operate subnets, deploy applications, and participate in consensus.

USDC is the payment currency. Workflow execution fees are denominated and settled in USDC, providing cost predictability for businesses using the protocol.

14.3 Collateral Lifecycle

Active participation in the network requires staked collateral. Tokens are locked for the duration of the participant's role and returned on clean exit (minus any penalties for validators who are slashed for provable misbehavior).

When a partner exits the network and forfeits their remaining allocation, those tokens are re-distributed to active participants as part of the network's collateral maintenance process. This redistribution is a consequence of the partner agreement structure described in the Token Utility

section.

The total supply is fixed at genesis. No minting occurs after the Token Generation Event. The Foundation Treasury allocation funds ecosystem development and is governed by the Foundation.

15 Security

15.1 BFT Assumptions and Limits

W3.io's security model rests on the standard BFT assumption: fewer than one-third of the validators in any given committee are Byzantine. For a committee of size n, the protocol tolerates up to f = floor((n-1)/3) faulty members. This is a well-understood bound. No BFT protocol survives a 2/3+ adversary.

The protocol assumes a partially synchronous network model: messages between honest validators are delivered within an unknown but finite bound. Under this model, the protocol guarantees safety (no two honest members reach conflicting decisions) and liveness (decisions are eventually reached, via view change if necessary). In a fully asynchronous network where messages can be delayed indefinitely, liveness cannot be guaranteed (by the FLP impossibility result), though safety is maintained. Beyond the $f < n/3$ bound, all bets are off. The protocol does not claim to defend against a supermajority adversary. No system can.

15.2 Attack Scenarios

The protocol's threat model considers several attack classes.

Faulty leader. A leader that fails to propose, proposes invalid values, or equivocates (sends different proposals to different members). Defense: automatic view change rotates leadership after timeout. Equivocation is detectable because members share received proposals, and provable on-chain for slashing.

Network partition. A subset of validators cannot communicate with the rest. Defense: the consensus protocol's liveness property requires only 2f+1 reachable members. A partition that isolates fewer than f+1 members does not prevent consensus. A larger partition stalls consensus until the partition heals, but does not produce incorrect results.

Trigger spoofing. An attacker claims a blockchain event occurred when it did not. Defense: monitor groups (7 members, f=2) independently verify trigger evidence. A single validator's claim is not sufficient. The monitor group runs BFT consensus to reach agreement on the trigger before dispatching execution.

Bad epoch root. An attacker attempts to submit a false epoch commitment to the L1 contract.

Defense: epoch submission requires a BLS quorum signature from 2f+1 committee members over the epoch tuple (chain ID, contract address, epoch number, validator set version, cumulative root, previous root, receipt count, namespace count). A single Byzantine validator cannot produce a valid quorum signature.

Replay. An attacker resubmits a previously valid epoch. Defense: the contract enforces monotonically increasing epoch numbers and root chaining. Resubmitting epoch N when the contract is past epoch N fails because the epoch number is not greater than the last accepted epoch.

Gossipsub manipulation. An attacker floods the gossipsub network with invalid messages or withholds valid messages. Defense: gossipsub v1.1 (Vyzovitis et al. 2020) includes peer scoring, message validation, and flood publishing. Invalid messages are rejected at the protocol level before reaching consensus.

15.3 Validator Set Security

The permissioned validator model provides an additional security layer beyond the BFT assumptions. Every validator operator is KYC-verified, meaning attack attribution is possible. An attacker who equivocates or submits bad data is identifiable and legally accountable, not just slashable.

The `emergencyRemove` function allows the Foundation to remove a compromised validator immediately without a vote. This is a centralized power that exists because the alternative (waiting while a known bad actor participates in consensus) is worse. The power is scoped: it can only remove, not add. It is disclosed and tracked on-chain.

A known unmitigated attack: a compromised `EMERGENCY_ROLE` holder could iteratively call `emergencyRemove` to shrink the validator set until their own validators form a quorum. This is acknowledged. The mitigation is progressive decentralization: remove the privileged role entirely as the network matures. The v1 trust model is an explicit, temporary compromise documented in the decentralization roadmap.

15.4 Epoch Chain Integrity

The settlement layer's epoch chain has several structural integrity properties.

Monotonic epoch numbers. The contract rejects any epoch with a number not greater than the last accepted epoch. This prevents replay and reordering.

Root chaining. Each epoch submission includes the previous epoch's cumulative root. The contract verifies this matches the stored root. This prevents forking: two different epoch N submissions would need to reference the same previous root, but only one can be accepted.

Validator set versioning. Each epoch submission includes the validator set version that produced

it. After an emergency removal advances the set version, epochs signed by the old set are rejected even if the signatures are mathematically valid.

Idempotent submission. If two validators race to submit the same valid epoch, the first is accepted and the second reverts cheaply. No harm done.

These properties are enforced by the L1 contract, not by the validators. A compromised validator set cannot bypass them without also compromising the L1 chain itself.

16 Risk Factors

Participation in the W3.io network and holding the W3 token involve significant risks. The following is not exhaustive. Prospective participants should carefully consider these factors.

16.1 Technology Risk

W3.io's protocol is complex software. Despite testing and code review, smart contracts and protocol logic may contain undiscovered bugs that could result in loss of staked funds, incorrect workflow execution, or settlement failures. Third-party security audits are planned but have not yet been completed. The protocol depends on underlying infrastructure (Avalanche L1, libp2p, Docker) that may itself contain vulnerabilities.

16.2 Regulatory Risk

The regulatory environment for digital assets is evolving. The W3 token may be classified differently by different jurisdictions. Regulatory changes could restrict the token's availability, limit its utility, or impose compliance requirements that affect the protocol's operation. W3.io does not provide legal, tax, or investment advice. Participants should consult their own advisors.

16.3 Validator Set Risk

W3.io launches with a permissioned, KYC-verified validator set. The initial validator set is small relative to established blockchain networks. A smaller validator set is more susceptible to coordinated failure, collusion, or external pressure than a larger, permissionless set. The BFT security guarantees assume fewer than one-third of committee members are Byzantine. This assumption is stronger for larger committees and weaker for smaller ones.

16.4 Smart Contract Risk

The L1 settlement contracts (EpochSettlement, ValidatorSet, WorkflowRegistry, SMTVerifier) are upgradeable via UUPS proxy with timelock controls. While upgradeability allows bug fixes, it also

means the contract behavior can change after deployment. Participants should be aware that the contracts they interact with today may behave differently in the future, subject to timelock delays.

16.5 Market Risk

The W3 token may lose value. The token's utility is tied to the W3.io protocol's adoption and usage. If the protocol does not achieve sufficient adoption, the demand for the token may be insufficient to support its value. The token is not redeemable for any asset and has no price floor.

16.6 External Dependency Risk

W3.io workflows depend on external services provided by ecosystem partners (data providers, payment processors, oracle networks). The availability, accuracy, and continued operation of these services is outside W3.io's control. A partner service outage or discontinuation could affect workflows that depend on that partner's actions.

16.7 Execution Attestation, Not Computational Proof

W3.io provides consensus-attested execution. A BFT quorum of validators agrees on the result of each workflow step. This is distinct from computational integrity proofs (such as ZK proofs) where a mathematical proof guarantees the computation was performed correctly. For workflow steps that interact with external, non-deterministic services, committee members may be unable to independently verify the result and may attest based on the runner's reported output. Participants should understand this distinction when evaluating the protocol's security guarantees.

17 Roadmap

17.1 Current State

W3.io's protocol is implemented in Rust, open-source, and deployed on testnet. The testnet processes over 200,000 workflow executions per day across a multi-node validator network. The protocol includes:

- Workflow execution runtime with GHA-compatible YAML compiler
- BFT consensus engine with 4-state machine for per-step agreement
- P2P networking via libp2p with gossipsub and direct committee messaging
- SSZ wire format for all protocol messages
- Native blockchain actions for Ethereum, Solana, and Bitcoin
- Ecosystem partner actions (Space and Time, Chainalysis, Pyth, Circle, Stripe, Hyperbolic, Redis, email)

- Backend trait with Docker execution, timeout enforcement, and signal escalation
- BLS12-381 signing with domain separation and PopVerifiedKey type-state enforcement
- Sparse merkle tree library with two-level cumulative structure
- Write-ahead log for epoch accumulation with crash recovery
- WAL-to-SMT epoch pipeline for settlement processing
- Validator capability bitmaps with ChaCha20 committee selection
- Validator join/leave lifecycle with heartbeat liveness detection
- Trigger definition registry with deduplication and monitor group assignment
- Namespace model with hierarchical authority and payer inheritance
- L1 settlement contracts (EpochSettlement, ValidatorSet, WorkflowRegistry, SMTVerifier)
- BLS quorum-signed epoch submission pipeline
- End-to-end integration test suite

Creatorland, W3.io's anchor client, is deployed on testnet as a live enterprise workflow.

17.2 Post-v1 Development

The following extension points are designed into v1 interfaces and will be developed after the settlement layer is in production.

VRF committee election. Replace the deterministic ChaCha20-based committee selection with verifiable random function output. VRF provides cryptographic proof that the committee was selected correctly, removing the need to trust the seed derivation. Entry point: `ValidatorView::get_participants()`.

Progressive decentralization. Transition from Foundation-managed operations through dual authorization to fully on-chain staking mechanics. Remove privileged administrative roles over time. Entry point: role admin transfer.

Data availability layer. Integrate Storj or Celestia for long-term proof archival. Currently validators store all execution traces. A dedicated DA layer provides durability guarantees independent of validator set churn. Entry point: W3Uri resolver with remote storage backends.

Namespace auth v2. Per-workflow authorization policies and key-based auth beyond address-based control. Supports use cases where different workflows within the same namespace have different access rules. Entry point: `WorkflowRegistry.sol` auth functions.

Validator capability proofs. Replace self-declared capability bitmaps with proven capabilities. A validator proves it has an Ethereum WebSocket connection by signing a recent block hash. Entry point: `ValidatorView` capability filter with proof verification.

Zero-downtime protocol upgrades. Epoch-activated feature flags that allow breaking changes

(wire format, consensus parameters) to deploy without network downtime. Validators upgrade on their own schedule within a window. All nodes switch behavior at the same epoch boundary. Entry point: protocol-set activation epoch with dual code paths.

Censorship dispute and slashing. On-chain dispute mechanism where namespace owners can prove receipt censorship using signed namespace summaries. Slashing via signer bitmap attribution. Entry point: dispute contract consuming `GossipReceipt` and `NamespaceSummary` evidence.

Field-level execution proofs and the `w3:` syscall. Complete the proof path from on-chain epoch root to individual execution field values. The `w3:` step kind enables workflows to generate merkle proofs about their own consensus-attested execution data and submit those proofs to L1 contracts. This enables selective disclosure (prove which validator ran a step without revealing other fields) and on-chain composability (a DeFi contract gates an action on a specific field of a compliance workflow's result). Foundation: `w3io-proof` crate (PR #1507). Entry point: `w3:` runtime handler wired to real `ConsensusStepRun` data.

Firecracker execution backend. MicroVM isolation for stronger step execution guarantees. Each step executes in its own lightweight VM with a dedicated kernel. Entry point: Backend trait implementation alongside Docker.

References

Boneh, D., S. Gorbunov, R. Wahby, H. Wee, C. Wood, and Z. Zhang. 2022. *BLS Signatures*. Draft-irtf-cfrg-bls-signature-05. https://datatracker.ietf.org/doc/draft-irtf-cfrg-bls-signature/.

Buterin, Vitalik. 2014. *A Next-Generation Smart Contract and Decentralized Application Platform*. https://ethereum.org/en/whitepaper/.

Castro, Miguel, and Barbara Liskov. 1999. "Practical Byzantine Fault Tolerance." *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI)*. https://pmg.csail.mit.edu/papers/osdi99.pdf.

Foundation, Ethereum. 2019. *Simple Serialize (SSZ)*. https://ethereum.org/en/developers/docs/data-structures-and-encoding/ssz/.

Gao, Zhenhuan, Yuxuan Hu, and Qinfan Wu. 2021. "Jellyfish Merkle Tree." *Diem (Aptos) Technical Documentation*. https://developers.diem.com/docs/technical-papers/jellyfish-merkle-tree-paper/.

Matsakis, Niko, and Felix Klock. 2014. "The Rust Language." *ACM SIGAda Ada Letters* 34 (3): 103–4. https://dl.acm.org/doi/10.1145/2663171.2663188.

Nakamoto, Satoshi. 2008. *Bitcoin: A Peer-to-Peer Electronic Cash System*. https://bitcoin.org/bitcoin.pdf.

Nir, Y., and A. Langley. 2018. *ChaCha20 and Poly1305 for IETF Protocols*. RFC 8439. https://www.rfc-editor.org/rfc/rfc8439.

Song, Yee Jiun, and Robbert van Renesse. 2008. *Bosco: One-Step Byzantine Asynchronous Consensus*. Cornell University. https://www.cs.cornell.edu/projects/Quicksilver/public_pdfs/52180438.pdf.

Supranational. 2023. *Blst: Multilingual BLS12-381 Signature Library*. https://github.com/supranational/blst.

Vlasov, Alex, and Kelly Olson. 2020. *EIP-2537: Precompile for BLS12-381 Curve Operations*. Ethereum Improvement Proposals. https://eips.ethereum.org/EIPS/eip-2537.

Vyzovitis, Dimitris, Yusef Napora, Dirk McCormick, David Dias, and Yiannis Psaras. 2020. *GossipSub: Attack-Resilient Message Propagation in the Filecoin and ETH2.0 Networks*. https://arxiv.org/abs/2007.02754.

Yakovenko, Anatoly. 2018. *Solana: A New Architecture for a High Performance Blockchain*. https://solana.com/solana-whitepaper.pdf.